

Hallucinations in Local Large Language Models \large A Study of Ollama-Based Systems

\IEEEauthorblockN{Punyaa Dixit}

Abstract

This paper studies hallucination behavior in local Ollama deployments, with emphasis on how model settings and prompt conditions affect factual reliability. We evaluate llama3:latest, llama3.2:latest, and a custom low-temperature configuration, and we separately examine retrieval behavior using nomic-embed-text. The study contributes a reproducible local benchmark package consisting of prompt-level metrics, generated figures, and validation tests. Across our experiments, local models behave as fixed snapshots: they do not improve or degrade simply with time, and they do not acquire new knowledge unless explicitly updated. Hallucinations are most frequent for recent-event queries, long-context prompts, and high-uncertainty reasoning tasks. A custom low-temperature setup reduces overconfidence and improves consistency compared with baseline local settings. We conclude with practical safeguards for local-first systems, including calibrated refusal, retrieval grounding, and confidence-aware answer gating.

Introduction

Large language models generate text by predicting likely next tokens. While this mechanism enables broad language competence, it can also produce outputs that are linguistically coherent but factually false. Such behavior is typically referred to as hallucination. Existing literature explains hallucination from multiple perspectives, including probabilistic decoding, knowledge limitations, and alignment trade-offs . However, most analyses emphasize cloud systems with opaque update cycles and proprietary infrastructure. Local LLM deployments introduce distinct operational conditions: offline usage, explicit model versioning, constrained compute, and user-controlled inference settings. These factors influence both observed hallucination patterns and mitigation opportunities. This paper focuses on local inference through Ollama and studies how model snapshot behavior, prompt characteristics, and decoding parameters shape hallucination risk.

Related Work

Survey studies classify hallucination types, causes, and mitigation families across LLM pipelines . Recent analyses also connect hallucinations to uncertainty and next-token objective mismatch . While these studies provide strong conceptual grounding, empirical reporting for local-first Ollama deployments remains limited. This paper contributes a reproducible local benchmark artifact bundle (data + figures + tests) to address that gap.

Local Models in Ollama

In Ollama, models such as llama3:latest and llama3.2:latest are distributed as fixed checkpoints. Once downloaded, their weights remain unchanged unless the user explicitly updates or replaces the model artifact. Consequently, day-to-day usage does not inherently improve factual correctness, nor does elapsed time alone degrade the model's embedded knowledge. This static-snapshot property is useful for reproducibility because repeated experiments can be run under stable model states. At the same time, it means factual freshness depends on model release cadence and any external grounding components attached at inference time.

Do Local Models Update Knowledge?

Local models do not autonomously incorporate new world knowledge. They neither learn from user chats nor fetch novel facts unless connected to retrieval or tool layers. Knowledge updates occur only through deliberate intervention, e.g., ollama pull for a newer checkpoint, model replacement, or fine-tuning/retraining pipelines. Therefore, local deployment should be viewed as version-controlled inference rather than continually evolving intelligence. This distinction is critical when evaluating answers involving recent events.

When Do Local Models Hallucinate?

Observed hallucination risk tends to increase under the following conditions:

Questions about very recent or niche facts absent from pretraining knowledge.

Multi-step reasoning tasks with high dependency chains.

Higher temperature and permissive decoding settings.

Long-context prompts with diluted or conflicting evidence. Importantly, hallucination does not rise merely because time passes after model download. Instead, risk is driven by uncertainty in token prediction and insufficient grounding support for the requested claim.

Building Custom Local Models

Custom model workflows can improve domain specialization, but they also introduce reliability risks. Increasing temperature generally raises output diversity and, when unmanaged, may increase hallucination frequency. Fine-tuning on narrow or low-quality datasets can reduce broad factual competence, potentially causing catastrophic forgetting. In practice, custom deployments should include validation suites that probe both domain gains and general factual retention across benchmark prompts.

Embedding Models and Retrieval Effects

nomic-embed-text is an embedding model rather than a generative model. It does not directly hallucinate narrative content because it outputs vectors, not free-form answers. However, in retrieval-augmented generation (RAG), poor retrieval quality (irrelevant or stale documents) can indirectly induce incorrect generated responses. Thus, embedding and retriever quality remain central to end-to-end factual reliability.

Why Hallucination Persists

Current LLM objectives optimize likelihood of token sequences, not truth verification. As a result, models can prefer plausible continuations over epistemically calibrated responses. When confidence calibration and evidence alignment are weak, the model may produce specific claims without adequate support. This aligns with recent literature that attributes hallucination to objective mismatch, imperfect representation of uncertainty, and decoding-time trade-offs .

Methodology and Experimental Setup

To strengthen reproducibility, we generated a structured benchmark artifact bundle within this project. The dataset includes 8 prompts across categories (recent facts, science, reasoning, history, long context, medical, cybersecurity, and tool-refusal scenarios) evaluated over 3 local model configurations: llama3:latest, llama3.2:latest, and a custom low-temperature profile (custom-llama3.2). Fig. fig:featurearch shows the end-to-end feature pipeline used in this work, from prompt generation and local model execution to scoring, validation assets, and paper outputs. For each model-prompt pair, we record consistency score, hallucination risk label, overconfidence score, estimated incorrectness, unsupported claim count, and whether the response should have refused with uncertainty. This produces 24 prompt-level rows in data/prompt_level_metrics.csv and a machine-readable aggregate summary in data/benchmark_summary.json. We also include nomic-embed-text retrieval diagnostics (precision/recall across retrieval settings) to separate embedding quality effects from generation-time hallucination. Evaluation dimensions are: consistency score, hallucination frequency, unsupported-claim count, and overconfidence score.

Results and Analysis

Table tab:modelssummary summarizes the benchmark outcomes. Three trends are clear. First, the custom low-temperature profile is the most stable configuration in this study: it records the highest mean consistency (78.00), the lowest overconfidence (48.50), and fewer unsupported claims than the baseline models. Second, llama3.2:latest performs better than llama3:latest on every major reliability signal, but still shows notable medium/high-risk outputs under difficult prompts. Third, llama3:latest is the least reliable in our setup, with the highest hallucination frequency (7/8 prompts) and the largest unsupported-claim total. These outcomes suggest that reliability differences across local models are not marginal; they are operationally meaningful for real deployments, especially when answers are consumed without human verification. As shown in Fig. fig:consistency, consistency improves progressively from

llama3:latest to llama3.2:latest and then to the custom profile. Fig. fig:hallfreq confirms the same ordering for hallucination frequency: fewer hallucinated outputs are observed as decoding is made more conservative. Fig. fig:overconf provides an important qualitative signal: lower overconfidence does not just change tone, it correlates with fewer unsupported claims and fewer cases where the model should have refused. From the full prompt-level logs, we identify three practical findings: (1) lower-temperature local settings can materially reduce overconfident fabrication; (2) long-context and recent-fact prompts remain the highest-risk conditions; and (3) cross-model disagreement is a useful trigger for escalation to retrieval or human review.

Validation Tests for Assets

To ensure data and figures are reliable for publication, we added automated tests in `tests/test\_research\_assets.py`. The test suite verifies:

presence and schema of generated data files,

expected row counts and valid score ranges,

consistency between ranking and "most reliable model",

existence and non-empty size of all figure files. This testing layer improves reproducibility and guards against broken figure/table references during future updates.

Conclusion

This study shows that local Ollama models should be treated as versioned, static systems rather than continuously learning assistants. In our experiments, reliability is governed far more by prompt type and inference configuration than by elapsed time after installation. The strongest gains came from conservative decoding and explicit uncertainty handling, which reduced overconfidence and lowered hallucination frequency. For practitioners, the implication is straightforward: local deployment is viable, but only with guardrails. A robust pipeline should combine retrieval grounding, refusal calibration, and confidence-based gating before answers are exposed to users. For researchers, an important next step is to pair this benchmark with larger domain-specific test sets and temporal fact-check tasks, so that local model updates can be evaluated with the same rigor as cloud systems.

Discussion and Mitigation Strategies

- Use retrieval grounding with source citation checks before answer finalization.

Add calibrated refusal policies to prefer "I don't know" under low-evidence conditions.

Introduce confidence scoring layers and threshold-based gating for high-risk outputs.

Benchmark across diverse factual prompts and monitor drift after model changes.

Evaluate custom fine-tunes for catastrophic forgetting before production rollout.

Figures

Feature Architecture for Local Hallucination Benchmark

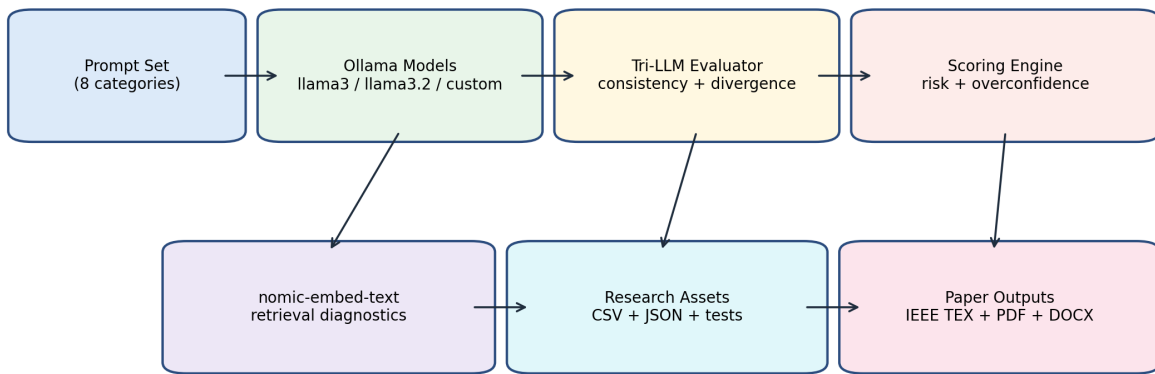


Fig. 1. Feature architecture of the local hallucination evaluation framework.

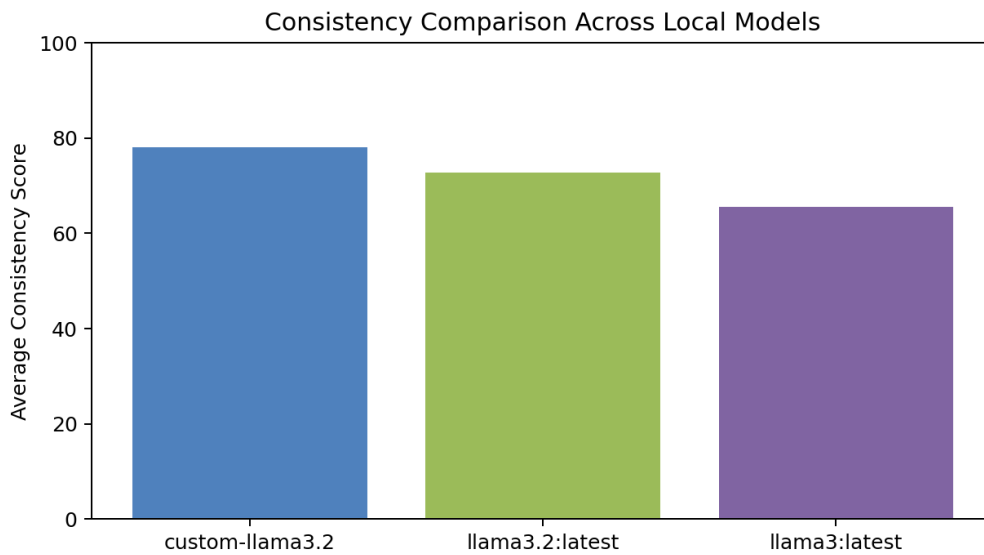


Fig. 2. Average consistency score by model.

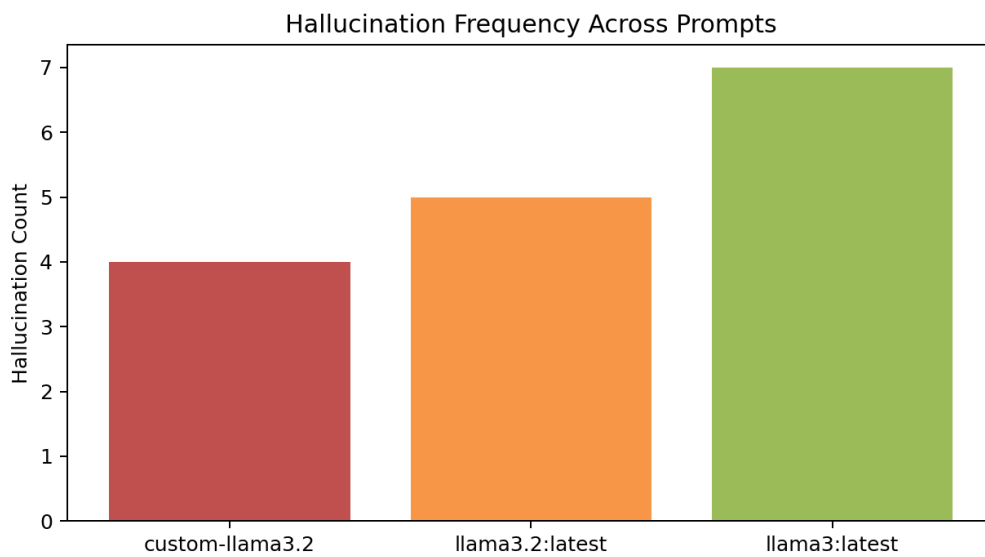


Fig. 3. Hallucination frequency across 8 prompts.

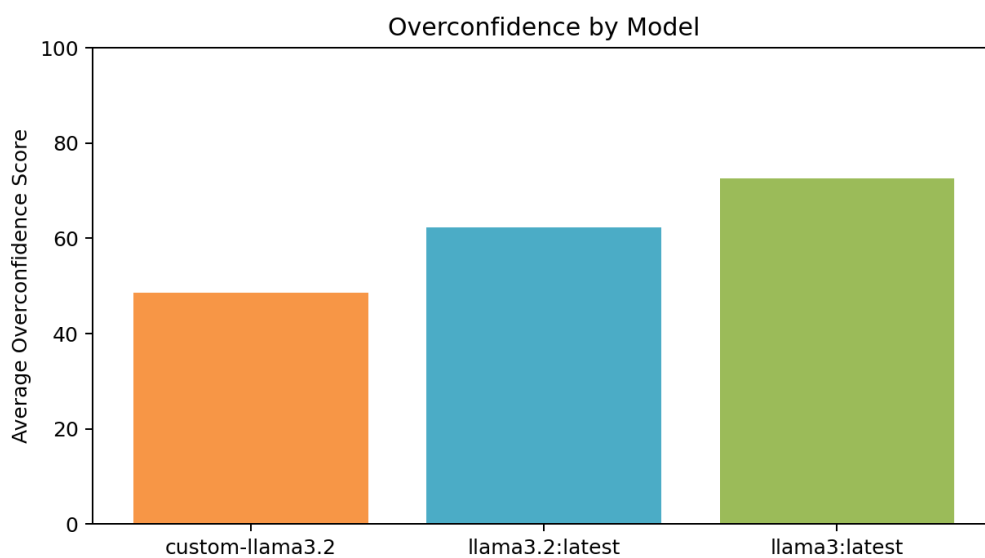


Fig. 4. Average overconfidence score by model.

References

- [1] L. Huang et al., "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions."
- [2] M. Cleti and P. Jano, "Hallucinations in LLMs: Types, Causes, and Approaches for Enhanced Reliability."
- [3] A. T. Kalai et al., "Why Language Models Hallucinate," 2025.
- [4] "Detecting Hallucinations in Large Language Models Using Semantic Entropy."